DAY 33 | HANDS-ON ACTIVITIES

LLM Deployment and Lifecycle: Problem Statements

Seven problems building a production-style LLM service. The model is simulated, so everything runs without a key or GPU.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Pick an Architecture

Level: Warm-up | **Concepts:** deployment options

Recommend an architecture for: a bank that cannot send data outside its tenant; a start-up prototype; a call centre with 50 million requests a month; a team using three different model providers.

Sample run / expected result

```
Bank: model hosted in its own cloud tenant
Start-up: hosted API
Call centre: evaluate self-hosting a fine-tuned small model
Three providers: AI gateway
```

Hints

- Ask about data rules, volume, skills and budget.

Problem 2: Fail-fast Settings

Level: Easy | **Concepts:** environments, secrets

Write settings loading for dev, test and prod. In prod, missing secrets must stop the service at startup with a clear message.

Sample run / expected result

```
APP_ENV=prod without LLM_API_KEY -> RuntimeError: LLM_API_KEY missing in prod
```

Hints

- Never print secret values in logs.

Problem 3: Timed Pipeline

Level: Medium | **Concepts:** inference pipeline

Build validate, enrich, generate, check and respond steps. Record the time of each stage and return it with the response.

Sample run / expected result

```
{"answer": "...", "timings_ms": {"validate": 0.1, "enrich": 12.4, "generate": 210.3, "check":
```

Hints

- time.perf_counter() around each step.

Problem 4: Concurrency Limit

Level: Medium | **Concepts:** asyncio, semaphore, 429

Simulate a provider that rejects more than 20 calls in flight. Send 100 requests unlimited and with a semaphore of 10, and compare errors and total time.

Sample run / expected result

```
unlimited: 80 rejected (429)
semaphore 10: 0 rejected
```

Hints

- asyncio.Semaphore(10) and asyncio.gather.

Problem 5: Streaming Endpoint

Level: Medium | **Concepts:** StreamingResponse

Add POST /chat/stream that yields words one at a time. Measure time to first chunk vs total time with TestClient streaming.

Sample run / expected result

```
first chunk after about 50 ms, full answer after about 220 ms
```

Hints

- TestClient buffers streams; measure with httpx.stream against a running uvicorn server.

Problem 6: Canary Routing

Level: Challenging | **Concepts:** canary, feature flags

Route 10% of users to release 1.9 based on a stable hash of their user id. Show that each user always gets the same release and roughly 10% get the canary.

Sample run / expected result

```
canary share: 9.8% of 5,000 users
user u-42 -> 1.8.0 every time
```

Hints

- Use hashlib, not Python's hash(), which changes between runs.

Problem 7: Rollback Rules

Level: Stretch | **Concepts:** release criteria

Given canary and baseline metrics, write a function that decides promote, hold or rollback using error rate, p95 latency and thumbs-down rate thresholds.

Sample run / expected result

```
error rate 3.1% vs 1.0% -> ROLLBACK
```

Hints

- Decide thresholds before the canary starts.

Nagareru (流れる): to flow